EXERCICE 08 · CHANGEMENTS ENTRE LES ÉTAPES

Formulaire d'enregistrement

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Point de départ** · template (3 fichiers, en entier)
2. **Étape 1 → 2** · correction (2 fichiers)

Départ Point de départ : template

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

Styles fournis (copiés tels quels, pas tapés en live) : src/App.css (195 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg, package.json

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 8 lignes

```jsx
import "./App.css";
import Form from "./components/Form";

function App() {
  return <Form />;
}

export default App;
```

DÉPART **src/components/Form.jsx** 64 lignes

```jsx
// Exercice 8 · template : l'interface est prête, à vous d'ajouter la validation (les TODO)
// TODO 1 : installer les librairies : npm install react-hook-form zod @hookform/resolvers
//          puis importer useForm, z et zodResolver

// TODO 2 : créer le schéma Zod
//   - fullName : 3 à 50 caractères, lettres uniquement
//   - email : format valide
//   - experience : nombre entier entre 0 et 10
//   - terms (bonus) : la case doit être cochée

function Form() {
  // TODO 3 : appeler useForm avec zodResolver(schema), les defaultValues et mode: "onSubmit"
  // TODO 4 : créer la fonction onSubmit(data), qui affiche les données dans la console

  return (
    <div className="container">
      <h1>Register Component</h1>
      <h2>Enter your information</h2>
```

```jsx

      {/* TODO 7 : si l'envoi a réussi (isSubmitSuccessful), afficher « Enregistrement OK » à la place du formulaire */}
      {/* TODO 5 : brancher le formulaire : onSubmit={handleSubmit(onSubmit)} */}
      <form name="login-form">
        <div className="form">
          <div className="form-group">
            <label>Full name</label>
            {/* TODO 5 : relier chaque champ avec register("...") */}
            <input type="text" />
            {/* TODO 6 : sous chaque champ, afficher le message d'erreur (errors.fullName.message) */}
          </div>

          <div className="form-group">
            <label>Email</label>
            <input type="text" />
          </div>

          <div className="form-group">
            <label>Experience</label>
            {/* Astuce : register("experience", { valueAsNumber: true }) pour avoir un nombre */}
            <input type="number" />
          </div>

          {/* Terms & Conditions (bonus) */}
          <div className="form-group">
            <label className="checkbox">
              <input type="checkbox" />
              <span>
                J'accepte les{" "}
                <a href="#" target="_blank" rel="noreferrer">
                  Conditions d'utilisation
                </a>
              </span>
            </label>
          </div>

          <button className="btn" type="submit">
            Register
          </button>
        </div>
      </form>
    </div>
  );
}

export default Form;
```

Étape 1 → 2 template → correction

La validation avec React Hook Form et Zod : le schéma, useForm avec zodResolver, register et les messages d'erreur.

MODIFIÉ `package.json` +4 −1

```
@@ -10,8 +10,11 @@
      "preview": "vite preview"
    },
    "dependencies": {
+     "@hookform/resolvers": "^5.2.2",
      "react": "^19.2.0",
-     "react-dom": "^19.2.0"
+     "react-dom": "^19.2.0",
+     "react-hook-form": "^7.68.0",
+     "zod": "^4.2.1"
    },
    "devDependencies": {
      "@eslint/js": "^9.39.1",
```

MODIFIÉ `src/components/Form.jsx` +98 −49

```
@@ -1,62 +1,111 @@
- // Exercice 8 · template : l'interface est prête, à vous d'ajouter la validation (les TODO)
- // TODO 1 : installer les librairies : npm install react-hook-form zod @hookform/resolvers
- //          puis importer useForm, z et zodResolver
+ // Exercice 8 · correction : le template + la validation avec React Hook Form et Zod
+ import { useForm } from "react-hook-form";
+ import { z } from "zod";
+ import { zodResolver } from "@hookform/resolvers/zod";

- // TODO 2 : créer le schéma Zod
- //   - fullName : 3 à 50 caractères, lettres uniquement
- //   - email : format valide
- //   - experience : nombre entier entre 0 et 10
- //   - terms (bonus) : la case doit être cochée
+ const schema = z.object({
+   // Champ "Nom complet"
+   fullName: z
+     .string()
+     .min(3, "Min 3 caractères")
+     .max(50, "Max 50 caractères")
+     .regex(/^[A-Za-zÀ-ÿ\s-]+$/, "Lettres uniquement"),
+   email: z.string().email("Email invalide"),
+   // Champ supplémentaire "Experience"
+   experience: z
+     .number()
+     .int("Nombre entier uniquement")
+     .min(0, "Min = 0")
+     .max(10, "Max = 10"),
+   // Checkbox "Terms & Conditions"
+   terms: z
+     .boolean()
+     .refine(
+       (v) => v === true,
+       "Vous devez accepter les Conditions d'utilisation"
+     ),
+ });

  function Form() {
-   // TODO 3 : appeler useForm avec zodResolver(schema), les defaultValues et mode: "onSubmit"
```

```
13      -   // TODO 4 : créer la fonction onSubmit(data), qui affiche les données dans la console
        30  +   const {
        31  +     register,
        32  +     handleSubmit,
        33  +     formState: { errors, isSubmitSuccessful, isValid },
        34  +   } = useForm({
        35  +     resolver: zodResolver(schema),
        36  +     defaultValues: { fullName: "", email: "", experience: 0, terms: false },
        37  +     mode: "onSubmit",
        38  +   });
        39  +
        40  +   const onSubmit = (data) => {
        41  +     console.log("Send to server:", data); // ici tu fais ton appel API
        42  +   };
14      43
15      44      return (
16      45        <div className="container">
17      46          <h1>Register Component</h1>
18      47          <h2>Enter your information</h2>
19      48
20      -       {/* TODO 7 : si l'envoi a réussi (isSubmitSuccessful), afficher « Enregistrement OK » à la place du formulaire */}
21      -       {/* TODO 5 : brancher le formulaire : onSubmit={handleSubmit(onSubmit)} */}
22      -       <form name="login-form">
23      -         <div className="form">
24      -           <div className="form-group">
25      -             <label>Full name</label>
26      -             {/* TODO 5 : relier chaque champ avec register("...") */}
27      -             <input type="text" />
28      -             {/* TODO 6 : sous chaque champ, afficher le message d'erreur (errors.fullName.message) */}
        49  +       {isSubmitSuccessful ? (
        50  +         <div className="success">Enregistrement OK</div>
        51  +       ) : (
        52  +         <form name="login-form" onSubmit={handleSubmit(onSubmit)}>
        53  +           <div className="form">
        54  +             <div className="form-group">
        55  +               <label>Full name</label>
        56  +               <input type="text" {...register("fullName")} />
        57  +               {errors.fullName && (
        58  +                 <p className="error">{errors.fullName.message}</p>
        59  +               )}
        60  +             </div>
        61  +
        62  +             <div className="form-group">
        63  +               <label>Email</label>
        64  +               <input type="text" {...register("email")} />
        65  +               {errors.email && (
        66  +                 <p className="error">{errors.email.message}</p>
        67  +               )}
        68  +             </div>
        69  +
        70  +             <div className="form-group">
        71  +               <label>Experience</label>
        72  +               <input
        73  +                 type="number" // Changé de "text" à "number"
        74  +                 {...register("experience", { valueAsNumber: true })} // Ajouté valueAsNumber
        75  +               />
        76  +               {errors.experience && (
        77  +                 <p className="error">{errors.experience.message}</p>
        78  +               )}
        79  +             </div>
        80  +
        81  +             {/* Terms & Conditions */}
```

```
+              <div className="form-group">
+                <label className="checkbox">
+                  <input type="checkbox" {...register("terms")} />
+                  <span>
+                    J’accepte les{" "}
+                    <a href="#" target="_blank" rel="noreferrer">
+                      Conditions d’utilisation
+                    </a>
+                  </span>
+                </label>
+                {errors.terms && (
+                  <p className="error">{errors.terms.message}</p>
+                )}
+              </div>
+
+              {Object.keys(errors).length > 0 && (
+                <p className="error">Veuillez corriger les champs invalides.</p>
+              )}
+
+              {isValid && <p className="success">Form valid (ready to send)</p>}
+
+              <button className="btn" type="submit">
+                Register
+              </button>
            </div>
-
-            <div className="form-group">
-              <label>Email</label>
-              <input type="text" />
-            </div>
-
-            <div className="form-group">
-              <label>Experience</label>
-              {/* Astuce : register("experience", { valueAsNumber: true }) pour avoir un nombre */}
-              <input type="number" />
-            </div>
-
-            {/* Terms & Conditions (bonus) */}
-            <div className="form-group">
-              <label className="checkbox">
-                <input type="checkbox" />
-                <span>
-                  J’accepte les{" "}
-                  <a href="#" target="_blank" rel="noreferrer">
-                    Conditions d’utilisation
-                  </a>
-                </span>
-              </label>
-            </div>
-
-            <button className="btn" type="submit">
-              Register
-            </button>
-          </div>
-        </form>
+          </form>
+        )}
      </div>
    );
  }
```